SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA,MANGALURU - 574146



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

FUNDAMENTALS OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

GROUP TASK REPORT ON

"BIG DATA APPLICATIONS, MACHINE LEARNING,BACKPROPAGATION AND CHATBOTS "

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ENGINEERING

Submitted By,

POORVI SUVARNA	01SU25AI055
DHANUSHREE S POOJARY	01SU25AI025
NIDHI NAGARAJ BHAT	01SU25AI050
SHRAVYA	01SU26AI703
GAGAN	01SU25AI028

UNDER THE GUIDANCE OF
PROF. MAHESH KUMAR V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

2025-2026

CONTENTS

Module 2 BIG DATA PROCESS MAPPING:GOOGLEMAPS

1.Introduction andSystem Selection
2.Big Data Characteristics of the System
3. Stage 1: Data Sources
4. Stage 2: Ingestion and Storage
5. Stage 3: Processing
6. Stage 4: Output
7.Worked Example: One Navigation Request
8. Governance, Privacy and Challenges
9. Conclusion

Module 3 MACHINE LEARNING PROCESS FLOW

1. Introduction and ProjectSelection
2. The Process Flow
3. Data Collection and Understanding
4. Data Preparation and Features
5.Algorithm Selection
6. Training, Validation, Testing and Evaluation
7.Deployment, Monitoring and Limitations
8. Conclusion

Module 4 BACKPROPAGATION WALKTHROUGH

1. IntroductionandObjective
2. Network, Notation and Starting Values
3. Forward Pass
4. Backward Pass and Gradient Flow
5.Weight Update
6. Verification and Training Behaviour
7. Observations and Conclusion

Module 5 RULE-BASED CHATBOT BUILD AND DEMO

1. Introduction and Objective
2.Architecture and Workflow
3. Intents, Rules and Context Logic
4.Mapping the Design to Dialogflow
5.Sample User Interaction
6.Testing and Observed Limitations
7. Improvements
8. Overall Conclusion and Future Scope

References

MODULE 2
BIG DATA PROCESS MAPPING: GOOGLE MAPS

Question (Module 2): *Big Data Process Mapping: Groups select a real-world big data system (like Google Maps, Amazon recommendations, or smart city sensors) and map out the entire data flow: data sources, storage, processing, and output.*

1. Introduction and System Selection

Big datarefersto datasets that are toolarge,too fast, or too varied for traditional single-machine tools, and to the methods used to collect, store, and analyse them. The group selected Google Maps as the real-world system because it uses nearly every kind of big data in one product: images of the whole planet, live position signals from millions of phones, structured road and transit records, and text contributed by users. It is also a system that every member of the group uses daily, which makes the output stage easy to observe.

One limitation must be stated at the outset. Google does not publish the internal design of its mapping systems, so the data flow in this module is a reasoned map built from public product descriptions, Google's published research, and standard big data practice. Where a stage is described in general terms, such as 'distributed storage', that is deliberate, because the specific internal products are not confirmed in public sources.

2. Big Data Characteristics of the System

Big datais commonlydescribedby asetof 'Vs', first articulated by Doug Laney as volume, velocity, and variety and later extended with veracity and value. Table 2.1 applies the five Vs to Google Maps.

Table 2.1: *The five Vs of big data applied to Google Maps*

Characteristic	Meaning	Evidence in google maps
Volume	Very large quantity of data	Imagery and road data for most of the world; historical traffic records for every major road.
Velocity	Data arrives and must be used quickly	Position signals from many phones arrive continuously, and traffic estimates must update within minutes.
Variety	Many formats and structures	Satellite images, street-level photographs, GPS points, transit timetables, business listings, reviews, and photographs.
Veracity	Data may be noisy or wrong	GPS error between tall buildings, outdated road data, and incorrect user edits must be detected and filtered.
Value	Data is useful only if it supports decisions	Faster journeys, accurate arrival times, discovery of places, and services built by other companies.

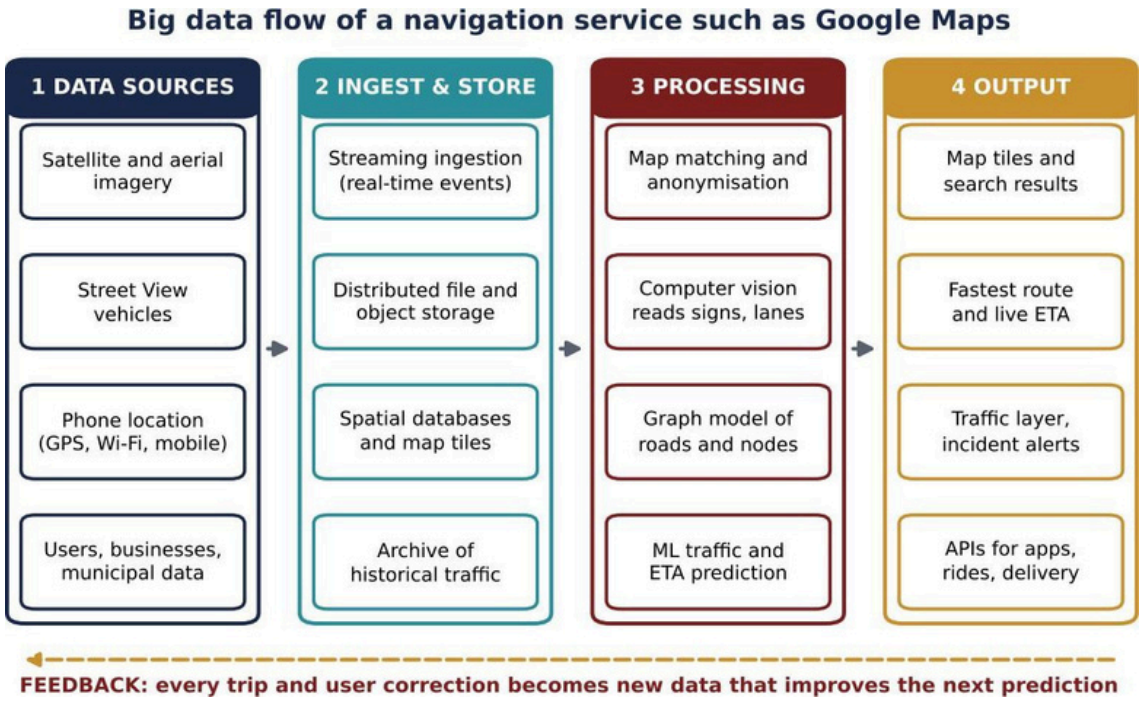


Figure 2.1: *Mapped data flow of a navigation service such as Google Maps: sources, ingestion and storage, processing, and output, with a feedback loop.*

3. Stage 1: Data Sources

The first stage gathers raw material from several independent sources. Aerial and satellite imagery gives thebasepictureof land, buildings, and roads. Camerason vehiclesand on portableequipmentcapture street-level imagery for Street View. Authoritative datasets from governments, transport agencies, and commercial providers supply road networks, speed limits, addresses, and public-transport timetables; the common timetable format used by transit agencies, GTFS, was in fact created by Google together with a transit agency. Business owners and ordinary users add place names, opening hours, photographs, reviews, and corrections.

The most time-sensitive source is the phone itself. Phones that have location services enabled and are using the app send anonymised location and speed information, which, when combined across many devices on the same road, shows how fast traffic is actually moving. Individually these signals are small; in aggregate they form a continuous high-velocity stream, which is why this source drives the velocity characteristic in Table 2.1.

4. Stage 2: Ingestion and Storage

Incoming data is separated by its nature. Live signals pass through a streaming ingestion layer that handles a constant flow of small events. Imagery and map files are loaded in batches into large distributed object storage, where files are replicated across machines so that the loss of a single disk or server does not lose data. Geographic information is held in spatial databases and indexed by area, so that a request for one neighbourhood can be answered without scanning the world. Map data is served in small square tiles at different zoom levels, so a phone downloads only what it needs for the current view. Historical traffic is archived because patterns repeat by hour and day of week, and that history later improves predictions.

5. Stage 3: Processing

Processing converts rawdatainto a usable model of the world and runs in two modes. In batch mode, large jobs build and update the base map. Computer vision models read street names, house numbers, and signs from imagery, and software joins data from different sources, resolving conflicts and removing duplicates. Privacy processing such as blurring faces and vehicle number plates in Street View also happens here.

In streaming mode, location signals are matched to roads and combined into live speed estimates. The road network is represented as a graph, with junctions as nodes and road segments as weighted edges, and route finding is a shortest-path search on that graph using algorithms of the Dijkstra and A* family with acceleration techniques for continental-scale networks. Google researchers have also described using graph neural networks to predict the time needed to travel along road segments, which improves arrival-time estimates by learning from historical and live traffic together. This is a direct use of the machine learning methods discussed in Modules 3 and 4.

6. Stage 4: Output

The output stage turns processed data into what users and other businesses see. These outputs are the map itself, search results for places, route options with a predicted arrival time, turn-by-turn guidance, a colour-coded traffic layer, and incident alerts. The same data is offered to external developers through an application programming interface, so ride-hailing, food-delivery, and logistics applications can use maps and routing inside their own products. The output is also the start of the next cycle: when many users slow down on a road, or take a different route from the one suggested, that behaviour becomes new input data. Table 2.2 summarises the whole flow.

Table 2.2: *Summary of the data flow in Google Maps*

Stage	Main activities	Data types	Technology category
1 Sources	Capture imagery; import official datasets; collect user and phone signals	Images, GPS points, tables, text	Cameras, satellites, mobile sensors, data partnerships
2 Ingestion and storage	Stream live events; load batches; index by location; archive history	Event streams, tiles, spatial records	Stream ingestion, distributed storage, spatial databases
3 Processing	Clean and merge data; read signs; build road graph; predict traffic and arrival time	Graphs, features, predictions	Batch and stream processing, computer vision, graph algorithms, machine learning
4 Output	Serve maps, routes, ETAs, traffic, and APIs; collect feedback	Tiles, routes, alerts	Web services, mobile apps, APIs

2.7 Worked Example: One Navigation Request

Table 2.3 follows a single request through the four stages to show how they work together in the few seconds between a person tapping 'Directions' and seeing a route.

Table 2.3: *A single navigation request mapped to the data flow stages*

Step	What happens	Stage involved
1	The user enters a destination; the app sends the start point, destination, and time of day.	Output (request) and sources
2	The routing service finds several candidate paths on the road graph held in storage.	Storage and processing
3	Live and historical speeds on each road segment are used to estimate the travel time of each path.	Processing (traffic model)
4	The fastest route and its arrival time are returned and drawn using map tiles.	Output
5	While driving, the phone keeps sending anonymised position and speed.	Sources (stream)
6	If the user leaves the route or traffic ahead slows down, the route is recalculated; aggregated speeds update the traffic layer for others.	Processing and output (feedback)

2.8 Governance, Privacy and Challenges

A locationtrail revealswhere a person lives, works, worships, and receives medical care, so governance is as important as engineering. The principles of informed consent, purpose limitation, and data minimisation apply directly to this kind of data. Figure 2.2 places the main controls at each stage. They include opt-in collection with clear prompts, controls that let users pause or delete their location history, anonymisation and aggregation before data is used for traffic, blurring of faces and number plates, and limits on what external developers can obtain through the API. In India the Digital Personal Data Protection Act, 2023 adds requirements of lawful consent and purpose limitation for processing of personal data.

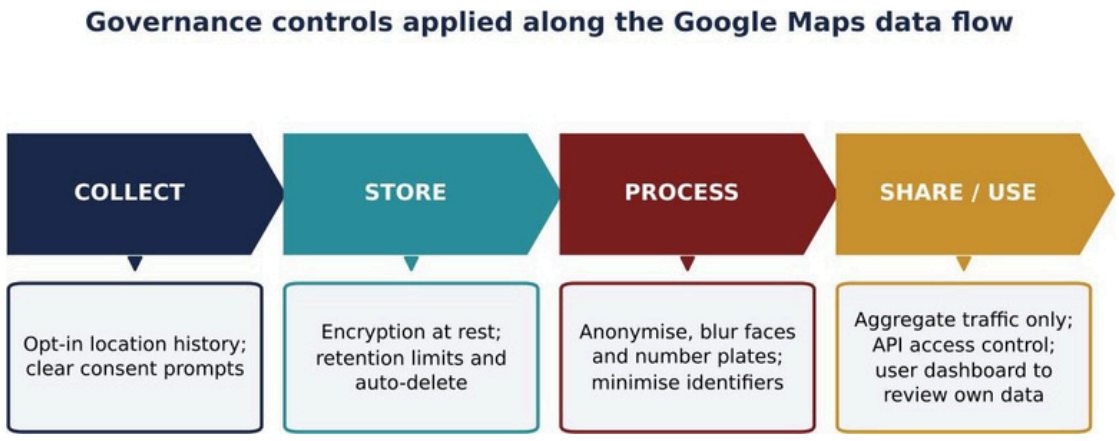


Figure 2.2: *Governance controls applied at each stage of the Google Maps data flow.*

Table 2.4: *Main challenges of the system and how they are reduced*

Challenge	why it occurs	How it is reduced
Inaccurate positions	GPS signals reflect off tall buildings and fail in tunnels.	Match positions to the road graph; combine with Wi-Fi and motion sensors; discard outliers.
Outdated map data	Roads, shops, and rules change faster than surveys.	Combine imagery updates, official data, and user reports; verify edits.
Privacy of travellers	Movement data is personal and sensitive.	Consent, anonymisation, aggregation, retention limits, and user controls
Scale and latency	Millions of requests need answers within seconds.	Precomputed tiles and route shortcuts, replicated storage, and distributed processing.
Bias in coverage	Data is richer where more people use the service.	Partnerships with local authorities and community mapping in under-served areas.

2.9 Conclusion

Mapping Google Maps showed that a big data system is a chain in which each stage depends on the others: sources decide what can be known, storage decides how quickly it can be retrieved, processing decides how useful it becomes, and output decides whether users keep generating new data. The feedback loop is the defining feature, because the product improves as people use it. The same loop is the reason for the governance controls in Figure 2.2, since every improvement in accuracy is paid for with data about people. The next module follows the machine learning part of such a pipeline in detail.

MODULE 3
MACHINE LEARNING PROCESS FLOW

Question (Module 3): *Build a Simple ML Process Flow: Groups create a complete flowchart for a machine learning project, covering data collection, feature extraction, algorithm selection, training, testing, and evaluation.*

1. Introduction and Project Selection

Amachine learning project is a sequence of decisions, and the quality of the final model depends on every step rather than on the algorithm alone. To show the whole sequence the group selected a small project that can be completed from start to finish: classifying iris flowers into three species from four measurements. The dataset was published by the statistician R. A. Fisher in 1936 and is distributed with the scikit-learn library. It is far cleaner than real business data, so the results below should be read as a demonstration of the process and not as a claim about difficult problems. All numbers in this module were produced by running the code on this dataset, with a fixed random seed so that they can be reproduced.

2. The Process Flow

The flowchart in Figure 3.1 has ten steps and one decision. The decision, whether the validated model is good enough, creates a loop back to earlier steps, because real projects rarely succeed on the first pass. Table 3.1 describes what happens at each step in the Iris project and what each step produces.

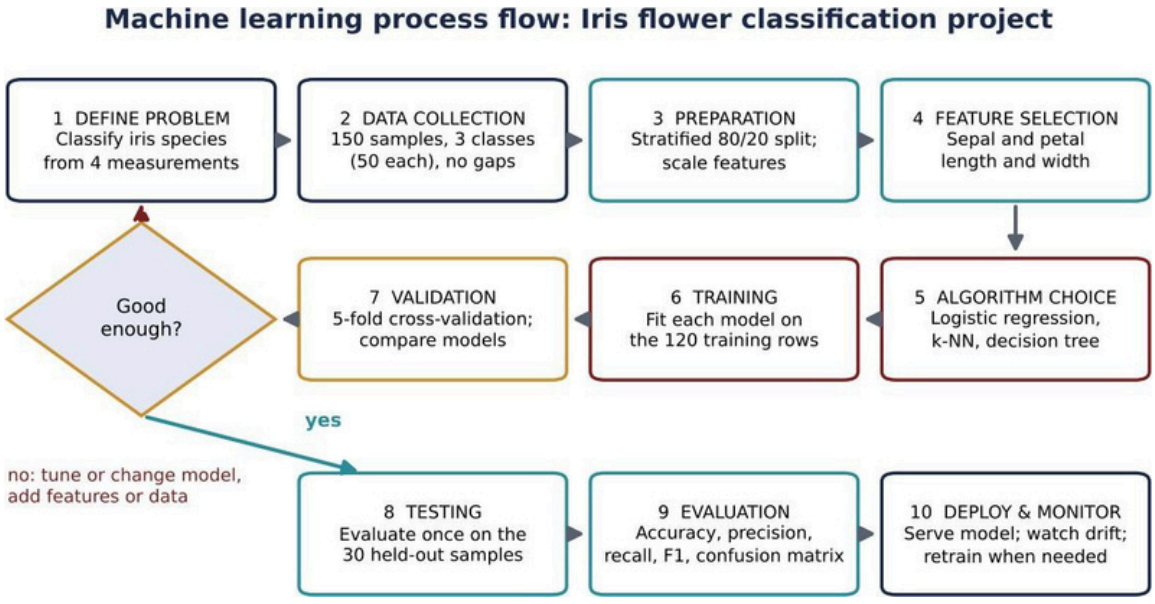


Figure 3.1: *Flowchart of the machine learning project, from problem definition to deployment, with a feedback loop after validation.*

Table 3.1: *Steps of the process flow applied to the Iris project*

Step	Activity in this project	Output
1 Define problem	Predict species (setosa, versicolor, virginica) from four flower measurements; this is supervised classification.	Problem statement and success measure (accuracy, per-class recall)
2 Data collection	Load 150 labelled samples, 50 per species; check for missing values and class balance	Raw dataset
3 Preparation	Split into 120 training and 30 test samples, keeping the species balanced; scale features using training data only.	Training and test sets
4 Feature selection	Use sepal length, sepal width, petal length, and petal width; the measurements are already numeric features.	Feature matrix
5 Algorithm choice	Shortlist logistic regression, k-nearest neighbours, and a shallow decision tree.	Candidate models
6 Training	Fit each candidate on the 120 training samples.	Trained models
7 Validation	Compare candidates using 5-fold cross-validation on the training set only.	Cross-validated accuracy
8 Testing	Evaluate the chosen model once on the 30 untouched test samples.	Test predictions
9 Evaluation	Compute accuracy, precision, recall, F1, and the confusion matrix.	Evaluation report
10 Deploy and monitor	Save the model, serve predictions, track input drift, and retrain when needed.	Maintained service

3.3 Data Collection and Understanding

The dataset contains150 rows with no missing values, and the three species are perfectly balanced at 50 samples each. Before any model is trained it is useful to look at how the classes differ. Table 3.2 gives the mean of each measurement for each species. Setosa has much shorter and narrower petals than the other two, so it is easy to separate. Versicolor and virginica overlap, with virginica having larger petals on average, and this overlap is where the errors reported later occur.

Table 3.2: *Mean of each measurement by species (centimetres), computed from the 150 samples*

Species	Sepal length	Sepal width	Petal length	Petal width
Setosa	5.01	3.43	1.46	0.25
Versicolor	5.94	2.77	4.26	1.33
Virginica	6.59	2.97	5.55	2.03

3.4 Data Preparation and Features

The data wasdivided with stratification, which means that each species keeps the same share in both parts: 40 training samples and 10 test samples per species. The test set is then locked away. Scaling puts all measurements on a similar range, which matters for distance-based methods and for logistic regression, and the scaling values are calculated from the training set only. Calculating them on the full dataset would let information from the test samples leak into training and make the final result look better than it should. This kind of leakage is one of the most common mistakes in real projects.

Feature extraction means turning raw data into numbers that a model can use, for example computing word counts from text or edge patterns from an image. Here the flower measurements are already

numeric, so extraction is trivial and the work is in choosing features. All four measurements were kept because there are few of them and each carries information, as Table 3.2 shows. With larger datasets, feature selection would also remove columns that are redundant or irrelevant.

3.5 Algorithm Selection

The problem has known labels and the output is a category, so supervised classification is the appropriate type of learning. Unsupervised methods would suit the case where species labels were not known, and reinforcement learning applies to sequential decisions with rewards, neither of which fits here. Three classification algorithms of different kinds were shortlisted, summarised in Table 3.3, so that the comparison would test different assumptions about the data.

Table 3.3: *Candidate algorithms and their main characteristics*

Algorithm	How it works	Strengths and weakness
Logistic regression	Learns a weighted sum of the features and converts it into class probabilities.	Simple, fast, gives probabilities; assumes roughly linear class boundaries.
k-nearest neighbours (k = 5)	Labels a sample by majority vote of the 5 closest training samples.	No training phase and flexible; slow on large data and sensitive to feature scale.
Decision tree (depth 3)	Learns a sequence of yes/no questions on the features.	Easy to read as rules; can overfit unless depth is limited.

3.6 Training, Validation, Testing and Evaluation

Each candidate was trained on the 120 training samples. To compare them without touching the test set, the group used stratified 5-fold cross-validation: the training data is split into five parts, the model is trained on four and checked on the fifth, and this is repeated so that every part is used once for checking. The average of the five scores is a more stable estimate than a single split. All three models obtained the same mean cross-validation accuracy of 95.8 per cent, with a standard deviation of 2.6 percentage points across folds, as shown in Table 3.4 and Figure 3.2.

Table 3.4: *Accuracy of the three models during validation and on the held-out test set*

Model	5-fold CV mean	CV std.dev	Test Accuracy
Logistic regression	95.8%	2.6 points	93.3% (28 of 30)
k-nearest neighbours (k = 5)	95.8%	2.6 points	93.3% (28 of 30)
Decision tree (depth 3)	\95.8%	2.6 points	96.7% (29 of 30)

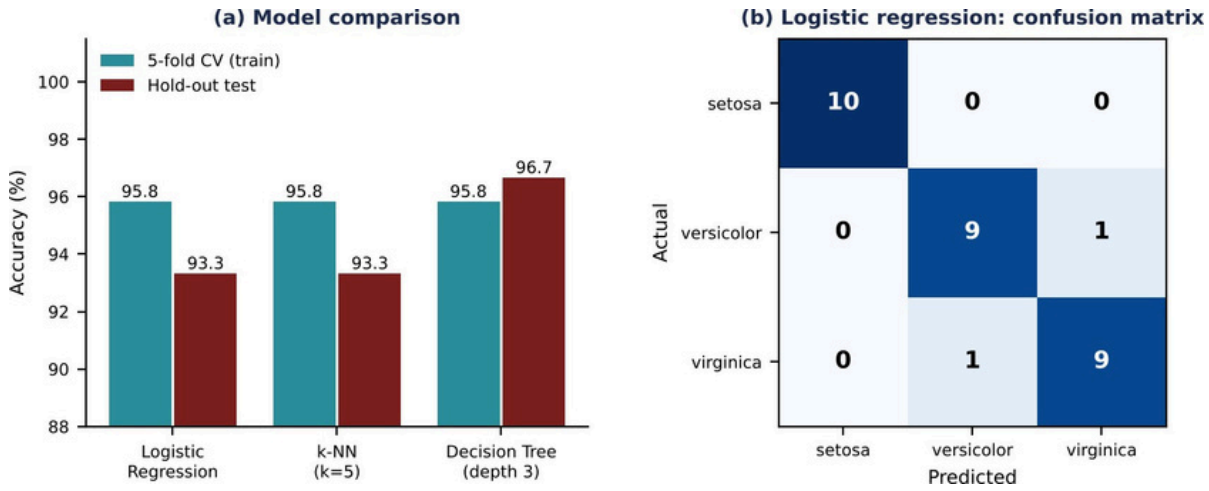


Figure 3.2: *(a) Cross-validation and test accuracy of the three models; (b) confusion matrix of logistic regression on the 30 test samples.*

Because validation could not separate the models, the group chose logistic regression, preferring the simplest model that gives class probabilities. This choice was made before looking at test results, and it was not changed afterwards. The decision tree scored one more correct test sample, but a difference of one sample out of 30 is within chance variation and switching to it after seeing the test set would turn the test set into a second validation set, defeating its purpose.

Evaluation used the confusion matrix in Figure 3.2(b). Logistic regression classified 28 of the 30 test samples correctly (93.3 per cent). All 10 setosa samples were correct. One versicolor was predicted as virginica and one virginica as versicolor, which are the two overlapping species noted in Section 3.3. Precision, recall, and F1 were 1.00 for setosa and 0.90 for each of the other two species. Accuracy alone would hide which class is being confused, which is why the per-class measures are reported as well.

3.7 Deployment, Monitoring and Limitations

In a deployed system the trained model and its scaling values would be saved together and exposed through a service that receives measurements and returns a species with a probability. After deployment the project continues: the distribution of incoming measurements is monitored for drift, a sample of predictions is checked against ground truth, and the model is retrained when performance falls. Table 3.5 lists common failure points and how this flow guards against each.

Table 3.5: *Common failure points in machine learning projects and the safeguard in the process flow*

Failure point	Consequence	Safeguard used here
Data leakage	Over-optimistic results	Scaling fitted on training data only; test set locked until step 8
Overfitting	Good training score, poor new-data score	Cross-validation; limited tree depth
Test-set reuse	Test score no longer honest	Model chosen by validation; test used once
Class imbalance	High accuracy that ignores a rare class	Stratified split; per-class precision and recall
No baseline	Unclear whether the model adds value	Three different models compared on the same folds

The main limitation of this demonstration is that the Iris data is small, clean, and balanced, and the test set contains only 30 samples, so each test error moves the accuracy by 3.3 points. Real projects face

missing values, imbalanced classes, shifting data, and fairness concerns, and they need larger test sets and more careful validation.

3.8 Conclusion

The process flow shows that machine learning is a disciplined workflow: define the problem, understand and split the data, choose candidate algorithms, validate them honestly, test once, evaluate in detail, and keep monitoring after deployment. On the Iris project all three models performed similarly, and the decision between them was made on principle and not by selecting the best-looking test number. The next module opens the training step and shows, with explicit arithmetic, how a neural network learns its weights.

MODULE 4
BACKPROPAGATION WALKTHROUGH

Question (Module 4): *Backpropagation Walkthrough: Together, step through a simple neural network's forward and backward pass calculations, showing gradient flow and weight updates.*

1. Introduction and Objective

A neural network learns by adjusting its weights so that its output moves closer to the correct answer. Backpropagation is the procedure that tells each weight in which direction and by how much to change. It applies the chain rule of calculus to compute the gradient of the error with respect to every weight in a single backward sweep through the network, and it was popularised for multi-layer networks by Rumelhart, Hinton, and Williams in 1986. Together with gradient descent it is the training method behind nearly all modern deep learning models, including convolutional and recurrent networks.

The objective of this module is to work through one complete training step by hand on a very small network, with every number shown, and then to check the arithmetic by computer. The values in all tables were calculated by a Python program (Appendix A), and the key gradient was verified independently by numerical differentiation.

2. Network, Notation and Starting Values

Thenetwork in Figure 4.1 has two inputs, two hidden neurons, and one output neuron. The weight w_ij connects input i to hidden neuron j, and w31 and w32 connect hidden neurons 1 and 2 to the output. Every neuron uses the sigmoid activation σ(z) = 1 / (1 + e^-z), whose derivative is σ(z)(1 - σ(z)). The error is measured by the squared error L = ½(t - y)², where t is the target and y is the output. The learning rate η is 0.5. Table 4.1 lists the starting values, which were chosen for easy hand calculation and not for any special property.

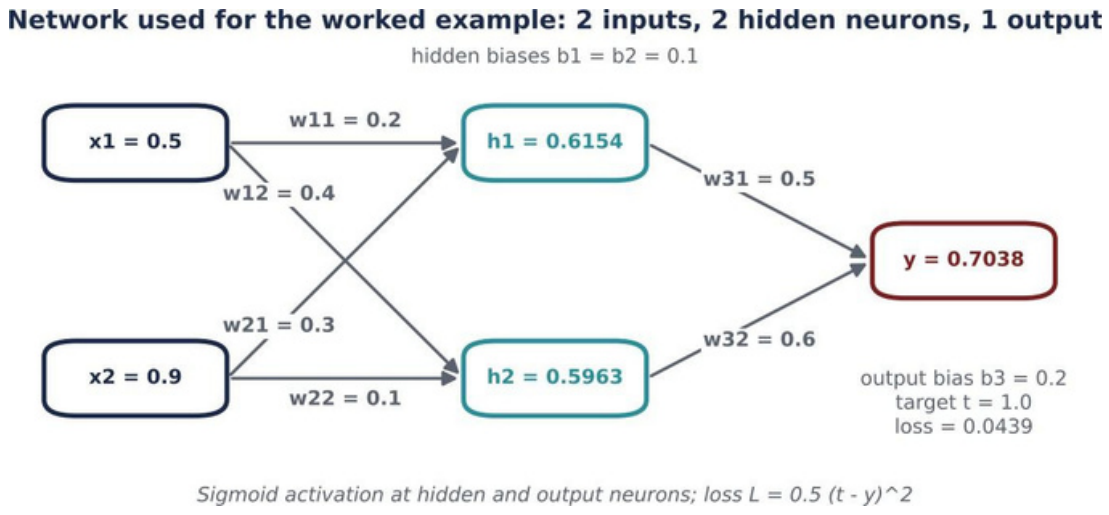


Figure 4.1: *Network used in the walkthrough, with initial weights and the activations obtained in the forward p a ss.*

Table 4.1: *Starting values of the walkthrough*

Quantity	Value
Input (x1, x2) and target t	x1 = 0.5, x2 = 0.9, t = 1.0
Input-to-hidden weights	w11 = 0.2, w12 = 0.4, w21 = 0.3, w22 = 0.1
Hidden biases	b1 = 0.1, b2 = 0.1
Hidden-to-output weights and bias	w31 = 0.5, w32 = 0.6, b3 = 0.2
Activation, loss, learning rate	Sigmoid; $L = \frac{1}{2}(t - y)^2$; $\eta = 0.5$

4.3 Forward Pass

Theforward pass computes the network's current output. Each neuron forms a weighted sum of its inputs plus a bias, called the pre-activation z, and then applies the sigmoid. Table 4.2 shows each calculation. With the starting weights the network outputs 0.7038 when the desired output is 1.0, so the loss is 0.0439 and there is room to improve.

Table 4.2: *Forward pass calculations*

Quantity	Calculation	Result
z1 (hidden 1)	0.5 × 0.2 + 0.9 × 0.3 + 0.1	0.4700
h1 = σ(z1)	1 / (1 + e^-0.47)	0.6154
z2 (hidden 2)	0.5 × 0.4 + 0.9 × 0.1 + 0.1	0.3900
h2 = σ(z2)	1 / (1 + e^-0.39)	0.5963
z3 (output)	0.6154 × 0.5 + 0.5963 × 0.6 + 0.2	0.8655
y = σ(z3)	1 / (1 + e^-0.8655)	0.7038
Loss L	½ × (1.0 - 0.7038)²	0.04387

4.4 Backward Pass and Gradient Flow

Thebackward pass starts at the loss and movestoward the inputs, reusing each result for the layer before it. First, the loss changes with the output as $\partial L/\partial y = y - t = -0.2962$. The output changes with its pre-activation as $\partial y/\partial z3 = y(1 - y) = 0.2085$. Multiplying gives the output delta $\delta 3 = -0.2962 \times 0.2085 = -0.0617$. The negative sign says that increasing z3 would reduce the loss.

The gradient for each output-layer weight is the delta times the input that weight received: $\partial L/\partial w31 = \delta 3 \times h1 = -0.0380$, $\partial L/\partial w32 = \delta 3 \times h2 = -0.0368$, and $\partial L/\partial b3 = \delta 3 = -0.0617$. To continue backward, the delta is passed to each hidden neuron through the weight that connects them and multiplied by the local sigmoid derivative: $\delta 1 = \delta 3 \times w31 \times h1(1 - h1) = -0.0617 \times 0.5 \times 0.2367 = -0.0073$, and $\delta 2 = \delta 3 \times w32 \times h2(1 - h2) = -0.0617 \times 0.6 \times 0.2407 = -0.0089$. The hidden-layer gradients follow in the same way, as shown in Table 4.3.

Table 4.3: *Backward pass: deltas and gradients (the arrows show the direction in which the error signal flows)*

Quantity	Calculation	Result
δ3 (output) ∂L/	(y - t) × y(1 - y) = -0.2962 × 0.2085	-0.06175
∂w31, ∂L/∂w32	δ3 × h1, δ3 × h2	-0.03800, -0.03682
∂L/∂b3	δ3	-0.06175
δ1 (hidden 1)	δ3 × w31 × h1(1 - h1)	-0.00731
δ2 (hidden 2) ∂L/	δ3 × w32 × h2(1 - h2)	-0.00892
∂w11, ∂L/∂w21	δ1 × x1, δ1 × x2	-0.00365, -0.00658
∂L/∂w12, ∂L/∂w22	δ2 × x1, δ2 × x2	-0.00446, -0.00803
∂L/∂b1, ∂L/∂b2	δ1, δ2	-0.00731, -0.00892

4.5 Weight Update

Gradientdescentmoves every parameter a small step against its gradient: new value = old value - η × gradient. Because allgradients here are negative, every parameterincreases. Table 4.4shows theupdate. When the forward pass is repeated with the new weights, the output rises from 0.7038 to 0.7153 and the loss falls from 0.04387 to 0.04053, a reduction of about 7.6 per cent from one step.

Table 4.4: *Weight update with learning rate η = 0.5*

Parameter	Old Value	Gradient	Change (-η × grad.)	new value
w11	0.2000	-0.00365	+0.00183	0.2018
w12	0.4000	-0.00446-	+0.00223	0.4022
w21	0.3000	0.00658	+0.00329	0.3033
w22	0.1000	-0.00803	+0.00401	0.1040
b1	0.1000	-0.00731	+0.00365	0.1037
b2	0.1000	-0.00892	+0.00446	0.1045
w31	0.5000	-0.03800	+0.01900	0.5190
w32	0.6000	-0.03682	+0.01841	0.6184
b3	0.2000	-0.06175	+0.03087	0.2309

4.6 Verification and Training Behaviour

Hand calculations can hidemistakes, so the group checked one gradient by numerical differentiation: w11 was nudged up and down by 0.000001, the loss was recalculated each time, and the slope was estimated from the two results. The estimate was -0.0036537, which agrees with the backpropagation value of -0.0036537 to all digits shown, confirming the chain-rule derivation. Repeating the update step on the same example moves the loss steadily downward, as Figure 4.2 shows: 0.0439 at the start, 0.0226 after 10 iterations, 0.0062 after 50, and 0.0014 after 200, at which point the output is 0.947.

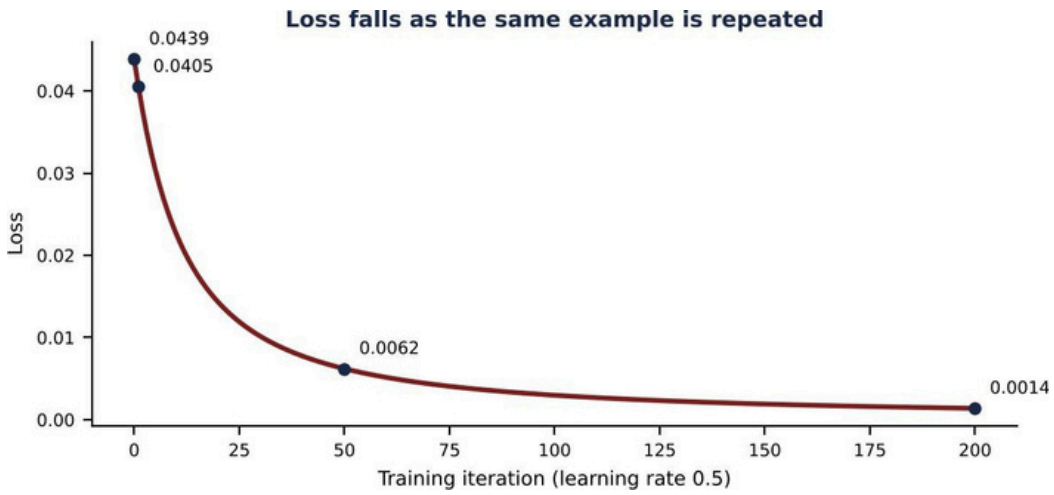


Figure 4.2: *Loss against training iteration when the update step is repeated on the single example.*

4.7 Observations and Conclusion

Three observations follow from the numbers. First, gradient flow shrinks as it moves backward. The output delta is -0.0617, while the hidden deltas are -0.0073 and -0.0089, about seven to eight times smaller, and the hidden-layer weight gradients are roughly five to ten times smaller than those of the output layer. The reason is that the sigmoid derivative never exceeds 0.25, so each layer multiplies the signal by a small number. In networks with many layers this repeated shrinking becomes the vanishing- gradient problem, which is why activation functions such as ReLU, residual connections, and gated units like the LSTM were developed.

Second, the size of each update depends on the learning rate. A step that is too small makes training slow, and one that is too large can overshoot the minimum, so practical training tunes this value. Third, real training is not done on a single example: gradients are averaged over mini-batches of data, optimisers such as Adam adapt the step size, and frameworks like PyTorch and TensorFlow compute all the derivatives automatically. Understanding the manual version, however, explains what those tools are doing and why training can fail. The same algorithm, extended through time, trains recurrent networks, and applied to shared filters it trains convolutional networks.

In conclusion, backpropagation is the chain rule applied systematically: a forward pass to measure the error, a backward pass to share the blame among the weights, and an update to reduce the error. The following module uses a trained model of a different kind, a conversation system, and examines its logic and limits.

MODULE 5
RULE-BASED CHATBOT BUILD AND DEMO

Question (Module 5): *Chatbot Build & Demo: Groups create a simple rule-based chatbot (e.g., with Dialogflow). Present its logic, user interactions, and improvements.*

1. Introduction and Objective

Achatbot is a program that holds aconversation in natural language. Rule-based chatbots follow explicit rules written by a designer, which makes them predictable, easy to test, and cheap to build, in contrast with learned models that generate replies from data. They remain widely used for frequently asked questions in customer service and education, where the range of questions is limited. The historical origin of the approach is ELIZA (1966), which was mentioned in Module 1.

The group built a chatbot called CampusBot that answers common student questions about timings, fees, examinations, and contacts. It was written in Python using regular expressions, so that every rule is visible, and it was designed using the same concepts as Dialogflow (intents, entities, contexts, and a default fallback) so that it could be rebuilt there. All timings, dates, and fee details in the bot are sample data created for this demonstration and do not describe any real institution.

2. Architecture and Workflow

The botworks in four stages, shown in Figure 5.1. The input is cleaned by converting it to lower case and trimming spaces. The intent matcher tests the cleaned text against a list of patterns in order, and the first match decides the intent. A small context memory stores what the bot asked or offered in the previous turn, so that a short reply such as 'yes' or 'canteen' can be understood. Finally a template reply is returned, or a fallback message if nothing matched.

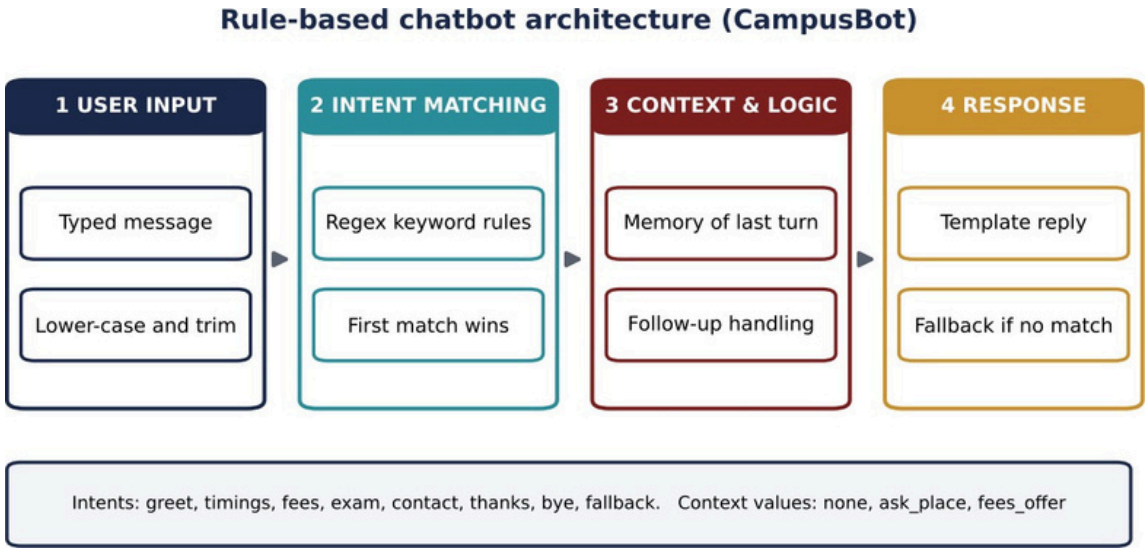


Figure 5.1: *Architecture of the rule-based chatbot: input, intent matching, context logic, and response.*

5.3 Intents, Rules and Context Logic

Anintentisthe goal behind a user's message. CampusBot has eight, listed in Table 5.1 with their trigger words. Patterns match whole words only, so the word 'fee' does not wrongly fire inside 'coffee'. The timings intent is the only one that needs extra information: if the message names a place (library, office,

or canteen) the answer is given directly, and if not the bot asks which place is meant and stores the context ask_place. The fees intent offers the due date and stores the context fees_offer, so a following 'yes' is treated as acceptance. Figure 5.2 shows this logic.

Table 5.1: *Intents, trigger patterns, and responses of CampusBot*

Intent	Trigger words (whole words)	Response
greet	hi, hello, hey, good morning/afternoon/evening	Greeting and list of topics
timings	time, timing(s), hours, open, close	Opening hours of the named place, or a question asking which place
fees	fee, fees, payment, tuition	Where to find fee details, then an offer of the due date
exam	exam, exams, timetable, schedule	Where the examination timetable is published
contact	contact, phone, email, reach, help desk	Help desk location and hours
thanks / bye	thanks, thank you / bye, goodbye, exit, quit	Polite closing replies
fallback	(no rule matched)	Apology and a list of topics the bot can handle

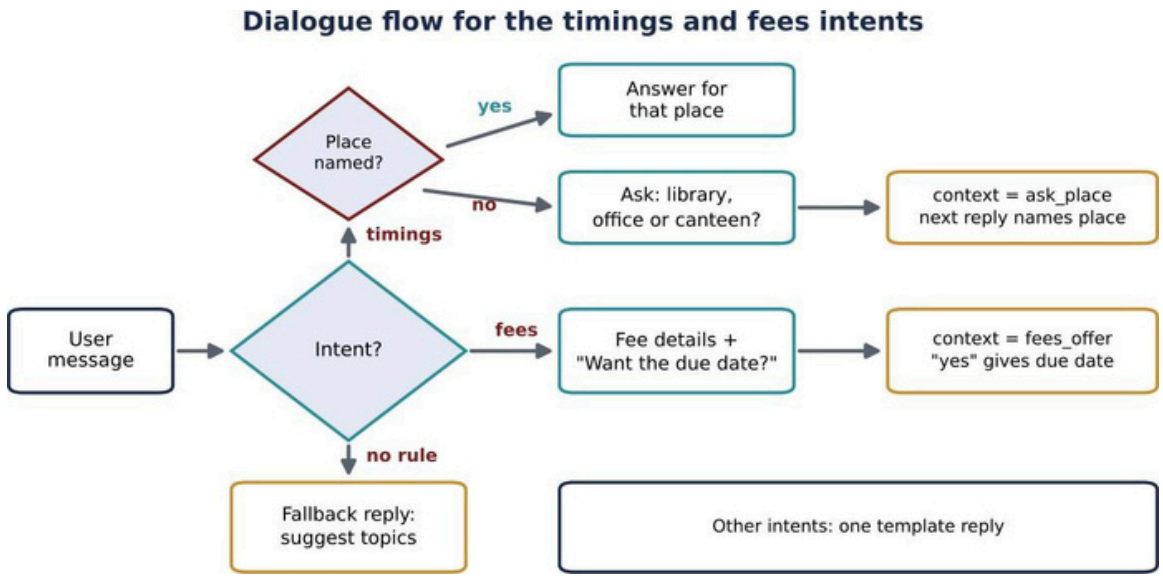


Figure 5.2: *Dialogue flow of the timings and fees intents, including the context values that carry meaning to the next turn.*

5.4 Mapping the Design to Dialogflow

Dialogflow is a Google Cloud service for building conversational agents. The same design can be created there without writing matching code, because each component of the Python bot has a direct counterpart, as Table 5.2 shows. The advantage of Dialogflow is that its intent matching is learned from example phrases and so tolerates different wordings, whereas a hand-written pattern must anticipate them.

Table 5.2: *Components of the Python bot and their Dialogflow equivalents*

CampusBot component	Dialogflow concept	
Regex rule per intent	Intent with training phrases	Dialogflow generalises from examples to similar sentences.
PLACES dictionary	Custom entity (for example @place)	Extracts library, office, or canteen from the sentence.
Context value (ask_place, fees_offer)	Input and output contexts	Carry information between turns.
Template reply	Intent response	Static text or generated by a webhook.
Fallback reply	Default Fallback Intent	Handles input that no intent matches.
Sample data in code	Fulfillment webhook	Would query a real database for timings and dates.

5.5 Sample User Interaction

Table5.3showsaconversationofnine turns exactly as produced by running the program, with the intentthat thebotrecognised ateach turn.Itdemonstratesadirectanswer,afollow-upthatdependson context, a clarifying question, and a fallback.

Table 5.3: *Conversation with CampusBot (actual program output, sample data)*

Speak	Utterance	Intent	Bot reply
User	Hello	greet	Hello! I am CampusBot. Ask me about timings, fees, exams or contacts.
User	What are the library timings?	timings	The library is open from 8 AM to 6 PM on weekdays.
User	When are the fees due?	fees	Fee details are on the student portal under Accounts. Do you want the due date?
User	yes	followup	Fees are due by the 10th of the first month of the semester .
User	What are the opening hours?	timings	Which place: library, office or canteen?
User	canteen	timings	The canteen is open from 8 AM to 5 PM.
User	Tell me about the hostel	fallback	Sorry, I did not understand. You can ask about timings, fees, exams or contacts.
User	Thanks	thanks	You are welcome!
User	Bye	bye	Goodbye! Have a great day.

5.6 Testing and Observed Limitations

To findthe weaknesses ofthe rules thegroup tested the bot with additional messages that are realistic but different from the sample conversation. The results in Table 5.4 are mixed, and the failures are the most useful part of the demonstration because each one points to a specific improvement.

Table 5.4: *Additional test messages and the actual behaviour of the bot*

Test message	Bot behaviour	Result	Cause
Library hours please	Gave library timings	Pass	Keywords matched.
FEES	Gave fee information	Pass	Input converted to lower case.
I paid my coffee bill	Fallback reply	Pass	Whole-word matching ignores 'fee' inside 'coffee'.
How do I open a bank account?	Asked 'Which place: library, office or canteen?'	Fail	The word 'open' is a timings keyword; no sense of topic
Hi, what are the exam dates?	Replied with a greeting only	Fail	Greeting rule is first in the list and the first match wins.
libary timings (typo)	Asked which place	Fail	No tolerance for spelling mistakes.
Is the canteen open on Sunday?	Gave the weekday timing	Fail	Day of the week is not understood.

Three of the seven additional tests passed and four failed. The failures show the characteristic weakness of rule-based systems: they work for the cases the designer imagined and break on variation, multiple intents in one sentence, and spelling errors. They also fail without warning, as in the 'bank account' example, where the bot confidently gives an irrelevant reply instead of admitting that it does not know.

5.7 Improvements

Each weakness in Table 5.4 suggests an improvement. The group ranked the proposals by the benefit they bring against the effort they need.

Score instead of first match. Count the matching keywords for every intent and choose the highest score, or detect two intents in one sentence, so that 'Hi, what are the exam dates?' is answered as an exam question.

Tolerate spelling errors. Compare words by edit distance or phonetic similarity so that 'libary' is recognised as 'library'.

Use a learned intent classifier. Train a model such as TF-IDF with logistic regression, as in Module 3, on example sentences for each intent, or use Dialogflow or Rasa. A classifier also gives a confidence score, so that low-confidence messages such as the bank-account question can be sent to the fallback.

Extract entities. Recognise days and dates, so that a question about Sunday is answered correctly.

Connect to live data. Replace stored text with a webhook that reads timetables and fee records, so that answers do not become outdated.

Improve conversation handling. Record the unmatched messages to find new intents, support Kannada and Hindi in addition to English, and offer a hand-over to a human staff member for questions the bot cannot answer.

Protect privacy. Do not collect personal details in the chat; if student records are connected, authenticate the user first and follow the data-minimisation practices described in Module 2. Progress should be measured with intent accuracy on a labelled set of test messages, the fallback rate, and the task completion rate, in the same way that classifiers in Module 3 are measured on held-out data.

5.8 Overall Conclusion and Future Scope

The five modules of this report move from the idea of machine intelligence to a working program. Module 1 showed how Turing defined computation and posed the question of machine thinking.

Module 2 mapped the flow of big data through a real system, from sources to output, with governance at each stage. Module 3 described the full machine learning workflow and demonstrated it with reproducible results. Module 4 opened the training step and worked through backpropagation with every number shown. Module 5 built a conversational program, the kind of system Turing's test imagined, and measured where its rules fail.

Future work could replace the hand-written rules of CampusBot with a classifier trained on collected student questions and evaluated with the methods of Module 3, connect it to real timetable data under proper consent and access controls, and extend the backpropagation example to a network trained on a real dataset. Taken together, the work shows that practical artificial intelligence depends on sound foundations, honest evaluation of results, and an understanding of the limits of every method used.

References

1. Turing, A. M., 'On Computable Numbers, with an Application to the Entscheidungsproblem,' Proceedings of the London Mathematical Society, Series 2, vol. 42, 1936, pp. 230-265.
2. Turing, A. M., 'Computing Machinery and Intelligence,' Mind, vol. 59, no. 236, 1950, pp. 433-460.
3. Turing, A. M., 'Intelligent Machinery,' report to the National Physical Laboratory, 1948 (first published 1968).
4. Turing, A. M., 'The Chemical Basis of Morphogenesis,' Philosophical Transactions of the Royal Society of London B, vol. 237, 1952, pp. 37-72.
5. Copeland, B. J. (ed.), The Essential Turing, Oxford University Press, 2004.
6. Hodges, A., Alan Turing: The Enigma, Princeton University Press, 2014.
7. Searle, J. R., 'Minds, Brains, and Programs,' Behavioral and Brain Sciences, vol. 3, no. 3, 1980, pp. 4 1 7 -424.
8. Weizenbaum, J., 'ELIZA: A Computer Program for the Study of Natural Language Communication between Man and Machine,' Communications of the ACM, vol. 9, no. 1, 1966, pp. 36-45.
9. Russell, S., and Norvig, P., Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2021.
10. Laney, D., '3D Data Management: Controlling Data Volume, Velocity, and Variety,' META Group research note, 2001.
11. Derrow-Pinion, A., et al., 'ETA Prediction with Graph Neural Networks in Google Maps,' Proc. ACM CIKM, 2021.
12. Digital Personal Data Protection Act, 2023, Government of India.
13. Fisher, R. A., 'The Use of Multiple Measurements in Taxonomic Problems,' Annals of Eugenics, vol. 7, no. 2, 1936, pp. 179-188.
14. Pedregosa, F., et al., 'Scikit-learn: Machine Learning in Python,' Journal of Machine Learning Research, vol. 12, 2011, pp. 2825-2830.
15. Geron, A., Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 3rd ed., O'Reilly Media, 2022.
16. Rumelhart, D. E., Hinton, G. E., and Williams, R. J., 'Learning Representations by Back-Propagating Errors,' Nature, vol. 323, 1986, pp. 533-536.
17. Goodfellow, I., Bengio, Y., and Courville, A., Deep Learning, MIT Press, 2016.
18. Nielsen, M., Neural Networks and Deep Learning, Determination Press, 2015.
19. Jurafsky, D., and Martin, J. H., Speech and Language Processing, 3rd ed. (draft), Stanford University.
20. Google Cloud, Dialogflow documentation, https://cloud.google.com/dialogflow/docs.